PPA Speech Therapy Suite

Technical Reference Documentation

Version 4 (March 2026)

Single-file React application for Primary Progressive Aphasia (PPA) speech therapy.

Runs as a Claude.ai artifact or locally via Vite. Powered by Claude Sonnet.

1. Architecture Overview

1.1 Technology Stack

Component	Details
Frontend framework	React 18 (JSX with hooks)
Build tool	Vite 5+
AI model	claude-sonnet-4-20250514 (Claude Sonnet 4)
API access	Direct browser fetch to api.anthropic.com
Primary persistence	Browser localStorage — clip metadata, SR state, all content
Binary persistence	Browser IndexedDB — local video file data (ppa_video_files)
Styling	Inline styles (no CSS framework)
Distribution	Single compiled bundle: ppa-speech-therapy-bundle.jsx

1.2 File Structure

The source consists of a main entry file, three module files, a shared utility library, and ten data files compiled into a single bundle by bundle.js.

```
ppa-speech-therapy_main.jsx     ← app entry + 7 inline modules
NamingModule.jsx                ← Naming Practice (SR engine, phoneme starter,
graphic picker)
SentenceBuilderModule.jsx       ← Sentence Builder module
ExportImportSystem.jsx          ← export / import / backup utilities
shared.jsx                      ← fetchAnthropicApi, CallAPI, ThinkingDots
bundle.js                       ← bundler script (Node.js)
data/
  config.js          CLAUDE_MODEL, SYSTEM_PROMPT
  tools.js           TOOLS sidebar navigation (9 modules)
  namingItems.js     NAMING_ITEMS (10 seed items)
  assessmentTasks.js ASSESSMENT_TASKS
  scripts.js         SCRIPTS (4 functional communication situations)
  repetitionItems.js REPETITION_LEVELS
  sentenceTasks.js   SENTENCE_COMPLETIONS, SENTENCE_CONSTRUCTIONS
  videoClips.js      VIDEO_CLIPS, EMOJI_OPTIONS, Q_TYPES, helpers
  sbWordBank.js      SB_NOUNS/VERBS/ADJECTIVES/etc.
  sbConjugation.js   IRREGULAR_PAST/PARTICIPLE, conjugateVerb()
```

1.3 Module Inventory

Module ID	Label	Function
therapist	AI Therapist	Multi-turn chat with Dr. Aria (Claude)
assessment	Assessment	Structured language evaluation tasks
naming	Naming Practice	Word-finding with SR-driven order, semantic/phonemic cues, image upload
repetition	Repetition	Tiered word and phrase repetition drills
scripts	Script Training	Functional communication phrase practice
sentence_builder	Sentence Builder	Visual drag-and-drop sentence construction
sentence	Sentence Work	Completion and construction exercises
video	Video Questions	YouTube & local-file video comprehension (WHO/WHAT/WHERE)
progress	Progress	Activity heatmap, AI report, admin settings

1.4 Bundler (bundle.js)

The bundler strips import/export statements from all source files, stitches them in dependency order, and writes a single self-contained JSX file. It verifies parse correctness using tsx.

- **Usage:** `node bundle.js [rootDir] [entryFile] [outputFile]`
- **Default output:** `ppa-speech-therapy-bundle.jsx`
- **Parse validation:** `runs tsx --no-cache on the output`
- **Emoji safety:** only escapes true parser-breaking characters (\u2028, \u2029, \uFEFF, \u200B, \u200C). Variation selectors (\uFE0F), ZWJ (\u200D), and keycap combiners (\u20E3) are preserved as real Unicode.

1.5 Dependency Load Order

The bundler resolves the following 14-file dependency order to avoid temporal dead zone (TDZ) errors:

```
 1. data/config.js           8. data/assessmentTasks.js
 2. data/tools.js            9. data/videoClips.js
 3. data/namingItems.js     10. data/sbWordBank.js
 4. data/scripts.js         11. data/sbConjugation.js
 5. data/repetitionItems.js 12. SentenceBuilderModule.jsx
```

```
6. data/sentenceTasks.js   13. NamingModule.jsx
7. ExportImportSystem.jsx  14. ppa-speech-therapy_3_final.jsx
```

2. API Integration

2.1 CallAPI Component

All Anthropic API calls pass through shared.jsx, which exports three utilities: fetchAnthropicApi (low-level async fetch helper), CallAPI (render-less React component), and ThinkingDots (loading indicator). CallAPI fires on mount, uses AbortController to cancel the in-flight request on unmount, and always delivers a non-empty string to onResult. All required headers are applied centrally in fetchAnthropicApi — no manual patch is needed after copying to a Vite project.

```
function CallAPI({ messages, onResult, onError, system = SYSTEM_PROMPT })

// Required Vite headers (must be added manually after copying bundle to Vite src/):
headers: {
  "Content-Type": "application/json",
  "x-api-key": import.meta.env.VITE_ANTHROPIC_API_KEY,
  "anthropic-version": "2023-06-01",
  "anthropic-dangerous-direct-browser-access": "true",
}
```

2.2 API Key Configuration

Environment	Configuration
Vite (local)	VITE_ANTHROPIC_API_KEY in .env at project root (not src/)
Claude.ai artifact	Handled by Claude.ai infrastructure — no key needed
Model	claude-sonnet-4-20250514 (defined in data/config.js)
Max tokens	1 000 per request

2.3 AI Usage Per Module

Module	AI Usage
AI Therapist	Multi-turn conversation with full SYSTEM_PROMPT as Dr. Aria
Assessment	Evaluates spoken/written responses with clinical prompts
Naming Practice	Post-attempt feedback (cueing level, error type analysis)
Video Questions	Generates WHO/WHAT/WHERE questions from video description

Module	AI Usage
	(admin); post-round feedback
Progress	Generates clinical narrative reports from session log data

3. Persistence

3.1 localStorage Keys

All localStorage keys use the ppa_ prefix. No server-side storage exists.

Key	Module	Contents
ppa_naming_items	Naming	Array of word items (custom + seeded)
ppa_naming_sr	Naming	Spaced repetition state — { [word]: { interval, dueDate, streak, lastResult, lastSeen } }
ppa_repetition_levels	Repetition	Array of drill levels
ppa_scripts	Script Training	Array of situation+phrases objects
ppa_sentence_completions	Sentence Work	Completion exercise items
ppa_sentence_constructions	Sentence Work	Construction exercise items
ppa_video_clips	Video Questions	Custom clip metadata — no video data (stored in IndexedDB)
ppa_video_patches	Video Questions	Question edits to built-in clips
ppa_sb_library	Sentence Builder	Saved custom sentence library
ppa_progress_YYYY-MM-DD	Progress	Daily activity log entries (one key per day)
ppa_progress_settings	Progress	Report period, email, module detail levels
ppa_export_snapshots	Export system	Content hashes per exported filename
ppa_known_files	Export system	Known .ppa filenames per module
ppa_last_backup	Backup system	{time, contentHash} of last .ppabak download

3.2 IndexedDB — Video File Storage

✨ **New in v4 — local video files stored in IndexedDB**

Local .mp4 files selected through the Video Questions admin panel are stored in IndexedDB rather than localStorage. localStorage has a ~5 MB quota; a single compressed video file easily exceeds it, causing a silent QuotaExceededError that left the clip visible in React state but lost on remount. IndexedDB has no practical size limit.

Property	Value
Database name	ppa_video_files
Object store	files
Key	clip id (string)
Value	{ id, dataUrl } — base64 data-URL of the video file
IDB version	1

Helper functions defined in ppa-speech-therapy_3_final.jsx:

```
videoIdb_open()                  → Promise<IDBDatabase>
videoIdb_save(id, dataUrl)       → Promise<void>
videoIdb_load(id)                → Promise<dataUrl | null>
videoIdb_loadAll(ids[])          → Promise<Map<id, dataUrl>>
videoIdb_delete(id)              → Promise<void>
```

3.3 Seed-on-Mount Pattern

Five modules write their initial state to localStorage on first mount (empty dependency array). This ensures a full backup taken immediately after first load captures complete data, not just sparse defaults.

```
// RepetitionModule, ScriptsModule, SentenceModule, VideoModule,
SentenceBuilderModule:
useEffect(() => { saveLevels(levels); }, []);
useEffect(() => { saveScripts(scripts); }, []);
useEffect(() => { saveCompletions(c); saveConstructions(x); }, []);
useEffect(() => { saveLibrary(library); }, []);
// Note: VideoModule no longer seeds video file data to localStorage (now IndexedDB)
```

3.4 Video Clip Normalisation

VideoModule normalises all clips on load and save via normaliseClip(). fileUrl is intentionally excluded from what is persisted to localStorage; instead an isLocalFile flag is stored. Video data is written to IndexedDB separately.

Field	Normalisation behaviour
id	Preserved; generated if missing
isCustom	Always true
title	Preserved; defaults to 'Untitled Clip'
description	Falls back to title if empty
thumbnail	Defaults to 🎬
difficulty	Defaults to 'medium'
questions	Padded to 3 with blank WHO/WHAT/WHERE questions
youtubeId	Preserved with startSeconds / stopSeconds
fileName	Preserved if present (display only)
isLocalFile	Set to true if clip has fileUrl or isLocalFile flag — signals VideoModule to fetch data from IndexedDB
fileUrl	NOT stored in localStorage — kept only in IndexedDB and injected into the runtime clip object via fileUrls state

4. Admin PIN System

4.1 Overview

Default PIN: **1234** — hardcoded as `ADMIN_PIN` in `ppa-speech-therapy_3_final.jsx`. To change it, update that constant and rebuild the bundle.

> 💡 **Changing the PIN**
>
> Edit `ADMIN_PIN = "1234"` in `ppa-speech-therapy_3_final.jsx`, then run `node bundle.js`.

4.2 Per-Module Admin Capabilities

Module	Admin capabilities
Naming Practice	Add/edit/delete custom naming items with graphics (emoji, URL, or uploaded photo); export/import .ppa files
Repetition	Add/edit/delete custom drill levels; export/import .ppa files
Script Training	Add/edit/delete custom situations and phrases; export/import .ppa files
Sentence Work	Add/edit/delete completions and constructions; export/import .ppa files
Sentence Builder	Manage saved sentence library; export/import .ppa files
Video Questions	Add custom clips (YouTube URL or local .mp4); AI question generation; edit built-in clip questions; export/import .ppa files
Progress	Configure report settings; download full backup; restore from backup; clear progress data

4.3 Dirty-Check on Admin Close

All modules with export/import support run a dirty check when closing the admin panel. If any custom items have unsaved export changes, a re-export dialog is shown before the panel closes.

```
const closeAdmin = () => {
  const snaps = ppaGetSnapshots();
  const dirty = customItems.filter(it => ppaIsItemDirty(it, snaps));
  if (dirty.length > 0) { setShowReexport(true); return; }
  setAdminOpen(false);
};
```

5. Naming Practice Module

5.1 Spaced Repetition Engine

NamingModule.jsx contains a PPA-adapted spaced repetition engine. Standard SM-2 assumes the learner improves over time and uses intervals that grow to weeks or months. PPA is a progressive neurological condition — word-finding can regress even after successful sessions. The engine uses four compensating mechanisms:

Mechanism	Implementation
Short intervals	MAX_INTERVAL = 5 days. Nothing is ever delayed beyond 5 days regardless of success streak.
Asymmetric multipliers	correct ×1.2 / space_cued ×0.9 / semantic_cued ×0.8 / phonemic_cued ×0.5 / failed → reset to 1 day
Regression decay on load	srApplyDecay() runs on every load. Items overdue by 2× their interval: interval cut 40 %. Overdue by 4×: cut 60 %. Ensures the schedule tightens automatically if sessions become less regular.
Within-session re-queuing	REQUEUE_RESULTS = {phonemic_cued, failed}. These items are re-inserted into the current session queue REQUEUE_GAP (4) positions ahead, mirroring clinical same-session repetition practice.

SR Constants

```
const SR_KEY       = "ppa_naming_sr";
const MAX_INTERVAL = 5;      // days — hard cap for PPA
const REQUEUE_GAP  = 4;      // positions ahead for within-session re-queue
const SR_FACTOR = {
  correct:       1.2,
  space_cued:    0.9,  // mild step back — used Space-key phoneme starter
  semantic_cued: 0.8,
  phonemic_cued: 0.5,
  failed:        0,   // special-cased: resets to 1 day
};
const REQUEUE_RESULTS = new Set(["phonemic_cued", "failed"]);
```

SR State Shape

Stored in localStorage key `ppa_naming_sr` as a plain object keyed by word:

```
{
  [word]: {
    interval:   number,   // current interval in days (1-5)
    dueDate:    string,   // ISO date string YYYY-MM-DD
    streak:     number,   // consecutive non-failed attempts
```

```
    lastResult: string,   // correct | semantic_cued | phonemic_cued | failed
    lastSeen:   string,   // ISO date of most recent practice
  }
}
```

SR Functions

Function	Behaviour
srLoad()	Reads ppa_naming_sr from localStorage; returns {} on error
srSave(state)	Writes state to ppa_naming_sr
srApplyDecay(state)	Applies regression decay for items that are overdue relative to their stored interval; returns updated state
srRecord(state, word, result)	Calculates next interval and due date; increments/resets streak; returns new state (does not save — caller must call srSave)
srRebuildFromHistory()	Scans all ppa_progress_YYYY-MM-DD keys in chronological order and replays naming events to construct an initial SR state for returning users
srLoadOrBootstrap()	Calls srLoad(); if key absent, calls srRebuildFromHistory(); then applies srApplyDecay() and persists; used as useState initialiser
srQueue(items, srState)	Returns ordered array of item indices: overdue (most overdue first), then new, then upcoming (soonest first)

Queue Priority and Within-Session Re-queue

The session queue is rebuilt on every next() call after srApplyDecay() is applied. If the current word triggered a REQUEUE_RESULTS result, it is spliced back into the queue at position currentPos + REQUEUE_GAP so the patient practises it again before moving too far ahead.

5.2 Graphic Picker

The GraphicPicker component in NamingModule.jsx provides four tabs for selecting a naming item's graphic:

Tab	Mechanism
🎨 Pick emoji	Categorised grid of emoji sets; click to select
⌨️ Type emoji	Free-text input for pasting any Unicode emoji or symbol

Tab	Mechanism
📁 Upload photo	Hidden <input type='file' accept='image/*,.heic,.heif'> triggered by a styled button and a drag-and-drop zone. Calls compressImageFile() to produce a JPEG data-URL stored in the item's graphic field.
🔗 Image URL	Paste an http/https URL; stored directly as the graphic value

isImageGraphic() Helper

Replaces scattered `startsWith("http")` checks throughout the file. Returns true for both http: URLs and data: base64 images:

```
function isImageGraphic(g) {
  return g?.startsWith?.("http") || g?.startsWith?.("data:");
}
```

compressImageFile() Helper

Resizes and JPEG-compresses a browser File object using an offscreen canvas. Ensures images stay well under localStorage limits.

```
function compressImageFile(file, maxSide = 240, quality = 0.75)
// → Promise<string>  (JPEG data-URL)
// maxSide: caps the longer dimension in pixels
// quality: JPEG quality 0-1
// Reads via FileReader → draws on canvas → exports toDataURL('image/jpeg', quality)
```

Drag-and-Drop

The upload drop zone handles `onDrop`, `onDragOver`, and `onDragLeave`. The `processImageFile(file)` helper is shared between the file input onChange and the drop handler, so both paths go through the same type check and compression.

5.3 Hint-Phase Answer Input

The response text input is now rendered in all three practice phases (show, semantic, phonemic). The input is wired to the same response state so any text typed in show phase carries over into the hint phases. Border colour changes to match the hint palette (amber for semantic, blue for phonemic) to give visual continuity.

5.4 Phoneme Starter (Space Key)

In the show phase, when the response input is empty, pressing Space reveals one additional character from the start of the target word. Each subsequent Space press adds one more

character (e.g. A…, AP…, APP…). The revealed prefix is displayed in a blue-tinted panel above the input; the input border changes to blue to signal the active starter. Pressing Space once all characters are revealed has no further effect.

State: phonemesRevealed (integer, 0–word.length) is initialised to 0 and reset by next(). Submitting “Got it!” with phonemesRevealed > 0 records the result as space_cued (SR_FACTOR ×0.9) rather than correct (×1.2). Space key interception is guarded by response === “” so normal space-separated typing in multi-word items is unaffected.

6. Video Questions Module

6.1 Built-in Clips

Clip	Configuration
vc1 — A Day at the Market	youtubeId: zr9leP_Dcm8, starts at 10s
vc2 — Eating an Apple	youtubeId: YE7VzlLtp-4, 84s–96s
vc3 — At the zoo	youtubeId: jNQXAC9IVRw, from 0s
vc4 — Winter Walk	youtubeId: eRsGyueVLvQ, 25s–40s

6.2 Custom Clip Fields

Field	Description
id	Generated: custom_${Date.now()}
isCustom	Always true for custom clips
title	User-provided
description	Used by AI to generate questions
thumbnail	Emoji from EMOJI_OPTIONS picker
difficulty	easy / medium / hard
questions	Array of 3: { type, icon, color, question, options[4], answer, hint }
youtubeId	11-char YouTube video ID (extracted from URL)
startSeconds	Playback start time (optional, default 0)
stopSeconds	Playback end time (optional)
fileName	Original filename for display only (local-file clips)
isLocalFile	true for local-file clips — VideoModule loads data from IndexedDB

6.3 Local File Storage Architecture

saveCustomClips() is async and accepts an optional pendingFileEntries parameter:

```
const saveCustomClips = async (next, pendingFileEntries = []) => {
  const normalised = next.map(normaliseClip); // strips fileUrl, sets isLocalFile
  for (const { id, fileUrl } of pendingFileEntries) {
    await videoIdb_save(id, fileUrl);          // 1. write video data to IndexedDB
  }
  if (pendingFileEntries.length > 0) {
    setFileUrls(prev => ({ ...prev, ...Object.fromEntries(...) }));  // 2. update
state
  }
  setCustomClips(normalised);                  // 3. update React state
  localStorage.setItem("ppa_video_clips", JSON.stringify(normalised)); // 4. save
metadata
};

// Convenience wrapper for edits/deletes/imports (no new video data):
const saveCustomClipsSync = (next) => { saveCustomClips(next, []); };
```

On VideoModule mount, all local-file clip data is loaded from IndexedDB and injected into the allClips memo via a fileUrls state object:

```
const [fileUrls, setFileUrls] = useState({});

useEffect(() => {
  const localIds = customClips.filter(c => c.isLocalFile).map(c => c.id);
  if (localIds.length === 0) return;
  videoIdb_loadAll(localIds).then(map => {
    if (map.size > 0) setFileUrls(prev =>
({ ...prev, ...Object.fromEntries(map) }));
  }).catch(() => {});
}, []); // once on mount

const allClips = useMemo(() => [
  ...VIDEO_CLIPS.map(c => builtInPatches[c.id] ? { ...c, ...builtInPatches[c.id] } :
c),
  ...customClips.map(c => c.isLocalFile && fileUrls[c.id] ? { ...c, fileUrl:
fileUrls[c.id] } : c),
], [customClips, builtInPatches, fileUrls]);
```

deleteCustomClip() removes both the metadata from localStorage and the video data from IndexedDB:

```
const deleteCustomClip = (id) => {
  const next = customClips.filter(c => c.id !== id);
```

```
  saveCustomClipsSync(next);
  videoIdb_delete(id).catch(() => {});
  setFileUrls(prev => { const u = { ...prev }; delete u[id]; return u; });
  ...
};
```

6.4 nextClip Navigation

The `nextClip()` function advances clipIdx to the next position (wrapping) and calls resetClipState():

```
const nextClip = () => {
  const next = (clipIdx + 1) % allClips.length;
  setClipIdx(next);
  resetClipState();
};
```

6.5 Question Types (Q_TYPES)

Type	Icon / Color	Default placeholder
who	👤 teal (#4E8B80)	Who is in the video?
what	🎬 purple (#9B7FB8)	What are they doing?
where	📍 red (#C07070)	Where does this happen?

6.6 Clip Selector Dropdown

The clip selector uses position: fixed with coordinates from getBoundingClientRect() to avoid clipping by the parent scroll container. Scroll and resize listeners reposition the dropdown while open.

```
// On dropdown open:
const rect = dropdownRef.current.getBoundingClientRect();
setDropdownRect(rect);
setDropdownOpen(true);

// Dropdown style:
position: 'fixed',
top: dropdownRect.bottom + 6,
left: dropdownRect.left,
width: dropdownRect.width,
maxHeight: '60vh',
```

```
overflowY: 'auto',
zIndex: 9999,
```

6.7 Embed URL Construction

```
const embedUrl = clip?.youtubeId
  ? `https://www.youtube-nocookie.com/embed/${clip.youtubeId}
      ?start=${clip.startSeconds || 0}
      ${clip.stopSeconds ? `&end=${clip.stopSeconds}` : ''}
      &rel=0&modestbranding=1&cc_load_policy=1`
  : null;
// Local-file clips use clip.fileUrl (injected from IndexedDB) as a <video src>.
```

7. Export / Import System (ExportImportSystem.jsx)

7.1 File Formats

Extension	Purpose
.ppa	Module-specific content export (JSON). Contains ppaExport: true, version, moduleId, exportedAt, and module payload.
.ppabak	Full application state backup (JSON). Contains ppaBak: true, version, createdAt, keyCount, and all ppa_* localStorage keys. Note: IndexedDB video file data is NOT included in .ppabak files.

⚠️ **Video file data not in .ppabak**

Local .mp4 video data stored in IndexedDB is not included in the full backup. .ppabak files contain clip metadata (title, questions, difficulty) but not the video binary. To back up local clips, use the .ppa export from the Video Questions admin panel while the clips are still accessible, or keep the original .mp4 files separately.

7.2 Module Export Payloads

Module ID	Payload structure
naming	{ items: [...] } — naming items with id, word, category, graphic, cues
repetition	{ levels: [...] } — custom drill levels
scripts	{ situations: [...] } — custom situations
sentence	{ completions: [...], constructions: [...] }
sentenceBuilder	{ items: [...] } — saved sentence library entries
video	{ clips: [...] } — clip metadata and questions; no video binary data

7.3 Snapshot / Dirty Tracking

Function	Behaviour
ppaItemId(item)	Returns item.id ?? item._id
ppaContentHash(item)	JSON of content fields (strips id, _id, _sourceFile, _builtin)

Function	Behaviour
ppaIsItemDirty(item, snaps)	True if item has _sourceFile and its hash differs from snapshot
ppaRecordExportInMemory(...)	Records snapshots; returns items with _sourceFile set
ppaHandleReexport(...)	Groups dirty items by target file; downloads each
ppaHandleImport(...)	Validates moduleId; records snapshots on import

7.4 Full Backup System

Function / Component	Behaviour
ppaDoBackup(hint)	Collects all ppa_* keys; downloads as hint-YYYY-MM-DD.ppabak; records hash
ppaDoRestore(file, onError)	Reads .ppabak; writes all keys to localStorage; reloads page
ppaBackupIsStale(days)	Returns true if content hash changed OR last backup older than days
ppaContentStateHash()	Fast hash across 10 content-bearing keys (excludes progress logs)
BackupRestorePanel	UI card with Download + Restore buttons and staleness status

8. Progress Module

8.1 Activity Logging

Every completed exercise calls addToLog(), which appends an entry to the daily localStorage key ppa_progress_YYYY-MM-DD. Entries are plain objects with at minimum { type, item, result, time }.

The Naming Practice module enriches its log entries with the word and cueing result, which the spaced repetition engine reads during srRebuildFromHistory() to bootstrap a schedule for returning users.

8.2 Progress Settings

Setting	Default / Options
defaultPeriod	7days (options: 7days, 14days, 30days, 90days)
emailAddress	For display in generated reports (not used to send email)
moduleDetail	Per-module level: 'detailed', 'summary', or 'none'

8.3 Default Module Detail Levels

Module	Default level
AI Therapist	summary
Naming Practice	detailed
Assessment	detailed
Repetition	detailed
Sentence Work	summary
Script Training	summary
Sentence Builder	none
Video Questions	summary

9. Deployment

9.1 Vite Local Installation

1. Create a Vite React project:

```
npm create vite@latest ppa-therapy -- --template react
```

2. Copy ppa-speech-therapy-bundle.jsx to src/
3. Replace src/App.jsx:

```
import App from './ppa-speech-therapy-bundle.jsx'; export default App;
```

4. Create .env at project root (not src/):

```
VITE_ANTHROPIC_API_KEY=sk-ant-api03-...
```

5. In the bundle, find function CallAPI and add the required headers (see Section 2.1).
6. Optional: add body { margin: 0; } to index.css.
7. Run:

```
npm install && npm run dev
```

> 💡 **No Header Patch Needed**
>
> All required Anthropic headers (x-api-key, anthropic-version, anthropic-dangerous-direct-browser-access) are applied centrally in fetchAnthropicApi inside shared.jsx. No manual patch is required after deploying to a Vite project — add VITE_ANTHROPIC_API_KEY to .env and the headers are applied automatically.

9.2 Claude.ai Artifact

The bundle can be pasted directly as a React artifact in Claude.ai. No API key or header modification is needed — the Claude.ai infrastructure handles authentication.

9.3 Rebuilding the Bundle

```
# From the directory containing all source files and the data/ folder:
node bundle.js

# Output: ppa-speech-therapy-bundle.jsx
# Validation: tsx parse check runs automatically

# After rebuild, re-apply the API headers patch in function CallAPI
# if deploying to Vite (not needed for Claude.ai artifact).
```

10. Implementation Notes & Resolved Issues

10.1 Video Clip Data Loss — RESOLVED

✅ **Fixed in v4**

Previously, adding a local .mp4 clip and then navigating to another module caused the clip to disappear. Root cause: FileReader.readAsDataURL() encodes video as a base64 string (10 MB → ~13 MB); localStorage.setItem() silently throws QuotaExceededError at ~5 MB. React state was updated (clip appeared), but localStorage was not written. On remount the module re-read the unchanged localStorage, and the clip was gone. Fix: video binary data is now stored in IndexedDB (no size limit); localStorage holds only metadata.

10.2 nextClip Not Defined — RESOLVED

✅ **Fixed in v4**

The `nextClip` function was referenced by the 'Next Video →' button on the results screen but was never defined. Added adjacent to `nextQuestion`.

10.3 Bundle Emoji Encoding

The bundler only escapes true parser-breaking characters (\u2028, \u2029, \uFEFF, \u200B, \u200C). Variation selectors (\uFE0F), ZWJ sequences (\u200D), and keycap combiners (\u20E3) are preserved as real Unicode so emoji render correctly.

10.4 TDZ Error Pattern

useEffect hooks whose dependency arrays reference a state variable declared later in the function body cause a Temporal Dead Zone error at runtime. All useEffect calls must be placed after the state declarations they depend on. The SR engine follows this by declaring all state before any effects.

10.5 Dropdown Clipping

Dropdowns using position: absolute are clipped by parent scroll containers (overflow-y: auto). All custom dropdowns use position: fixed with getBoundingClientRect() coordinates to escape the stacking context.

10.6 localStorage Size Limits

Browser localStorage is limited to approximately 5–10 MB per origin. The shift of video binary data to IndexedDB eliminates the primary vector for quota overruns. Remaining candidates: large volumes of progress data or many naming items with high-resolution uploaded images.

The compressImageFile() helper (240 px max side, 75 % JPEG quality) keeps each naming graphic under ~25 KB.

10.7 IndexedDB and .ppabak Backup

The full backup (.ppabak) collects all ppa_* localStorage keys. IndexedDB data (video files) is NOT included. This is a known limitation. Administrators who rely on local .mp4 clips should also export those clips via the Video Questions .ppa export, which bundles the clip metadata and questions, and retain the original video files separately.